

Robot Control Algorithms for Line Following

Nik Swoboda

`nswoboda@fi.upm.es`

Robótica y Percepción Computacional
Universidad Politécnica de Madrid

March 3, 2026

In this module we will:

- talk briefly about control systems in general
- discuss possible control systems for line following (with a focus on feedback control)

Control Systems - An Example (1)

A popular example of a control system is a **thermostat**.

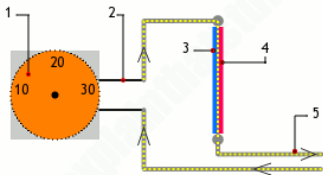
- it has a **goal state** (also **set-point** or **reference**), the desired temperature, which it tries to maintain
- it can measure **error**, the difference (positive or negative) between the current temperature and the desired temperature
- it generates commands, heat or no heat, to **achieve** and **maintain** the goal state.



Image from http://commons.wikimedia.org/wiki/File:Honeywell_thermostat.jpg

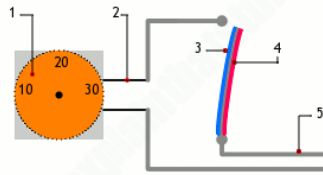
Control Systems - An Example (1)

A bimetallic thermostat (the simple heat or no heat type):



ON

www.explainthatstuff.com



OFF

www.explainthatstuff.com

blue=copper, red=steel; copper expands more than steel as it gets hotter

Image from <http://www.explainthatstuff.com/thermostats.html>

Control Systems - An Example (2)

A **Centrifugal Governor** (*Regulador Centrífugo*) on a steam engine

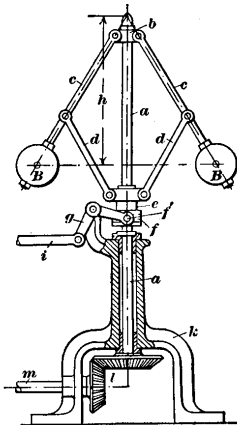


Video <https://www.youtube.com/watch?v=B01LgS8S5C8>

Image from http://commons.wikimedia.org/wiki/File:Georgetown_PowerPlant_Museum_109.jpg

Control Systems - An Example (2)

The centrifugal governor maintains a constant engine speed more or less independent of the load

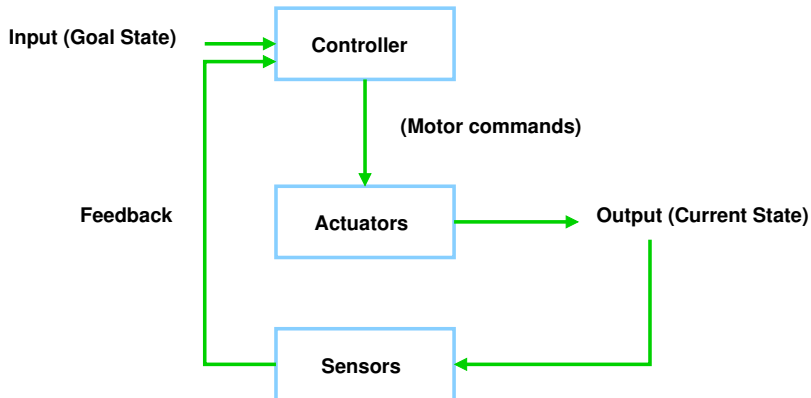


- B - Rotating weights (flyballs), as they move faster the distance h decreases
- m - drive from the engine
- i - linkage to throttle

Image from *Machine à Vapeur Horizontale* by Philip Taylor

Feedback Control

Feedback or Closed Loop Control (*Realimentación o Control de Lazo Cerrado*) is a simple mechanism used to help a system (robot) achieve and maintain a certain state. It works by constantly comparing the current state with the desired state and making corrections.



Feedback Control: A Real Life Example

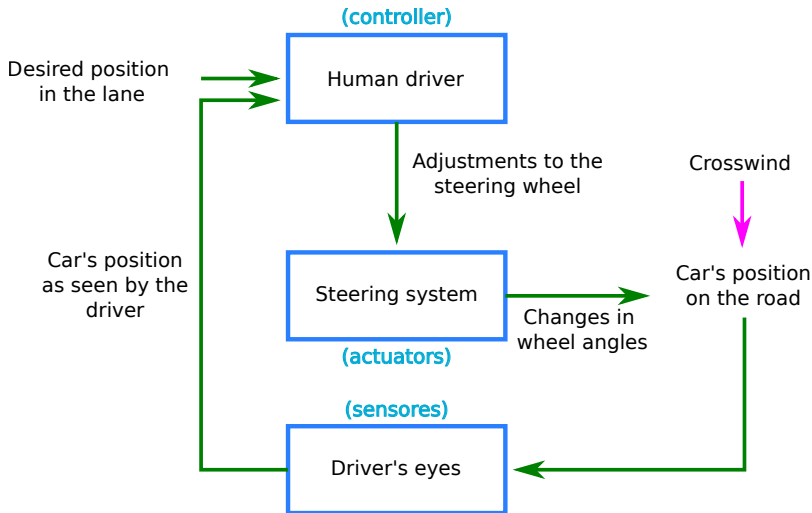


Image inspired by *Idiots Guide to PID Algorithms* by Finn Peacock

Some goals when designing feedback control systems:

- Stability (bounded disturbance produces bounded error)
- Good disturbance attenuation
- Speed of response to change
- ...

Can you classify this kind of control in terms of the control architectures that we saw last week? - Reactive.

We are going to focus on line following. What are some other applications in robotics for feedback control?

- wall following
- maintaining the balance of a biped robot
- keeping batteries in a certain range of charges

What are the drawbacks of using feedback control?

- possibility of instability (microphone feedback)
- additional measurement noise
- additional complexity

What is the 'opposite' of feedback control?

Feedforward or Open Loop Control (*Prealimentación o Control de Lazo Abierto*) does not use sensor data as feedback. In this system commands are determined based upon **predictions**.

This can work well when the environment is very predictable and does not change.

However, these are not very realistic assumptions for real applications of robotics.

But it can be interesting to combine feedback and feedforward control in the same system.

Feedback or Feedforward?

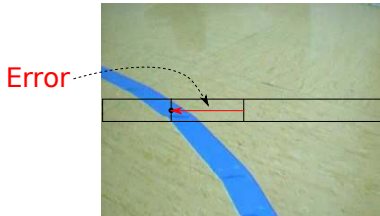


Images from <https://commons.wikimedia.org/wiki/File:Toaster.jpg>,
<https://commons.wikimedia.org/wiki/File:Oven.jpg>,
<https://commons.wikimedia.org/wiki/File:Washer.600pix.jpg>, and
https://commons.wikimedia.org/wiki/File:Ponderosa_elevator.JPG

Some Proposals for Line Following

The goal of a line following control system is to position and maintain a robot over a line while moving forward.

For the moment, we will measure **error** in terms of the distance between the center of the robot's field of vision and the center of the line which it is following.



Our goal is to determine the motor commands needed (**Forward Velocity** and **Turn Velocity**) to follow the line.

Some Proposals for Line Following

Why the 'for the moment'?



Later, after building your own vision system you can change the notion of error that your system uses.

Some Proposals for Line Following

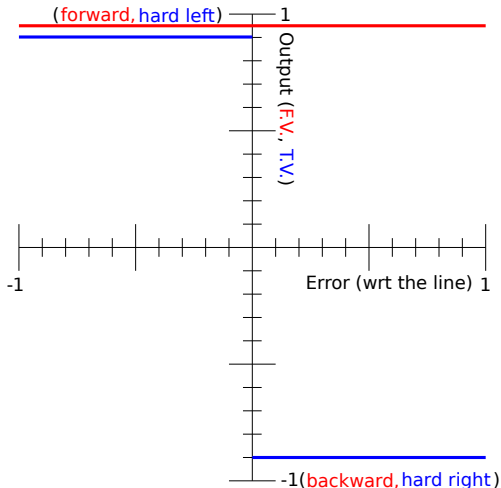
Any ideas regarding how to write a control system for line following?

Some Proposals for Line Following (1)

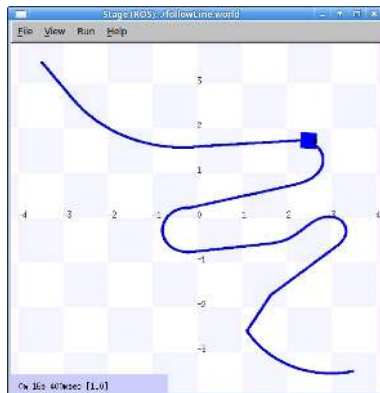
On-Off Control

Error	Turn Velocity
> 0	-0.9
0	0
< 0	0.9

Forward Vel.= .95



Some Proposals for Line Following (1)



Some Proposals for Line Following (1)

Comments:

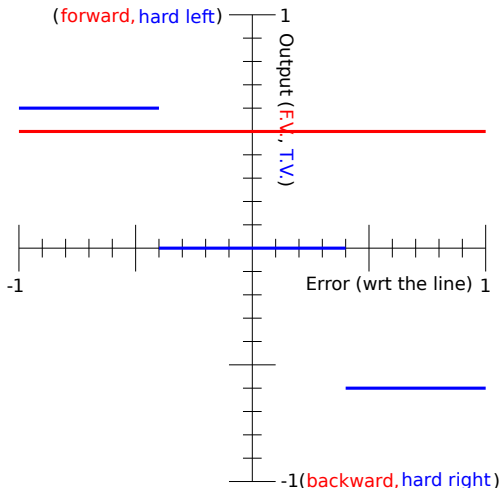
- It seems to be moving too quickly to handle sharp curves.
- Does it make sense to think of '0 error' as a state?
- Could we try something more flexible than just turn 0.95 or turn -0.95 ? (It seems to oscillate.)

Some Proposals for Line Following (2)

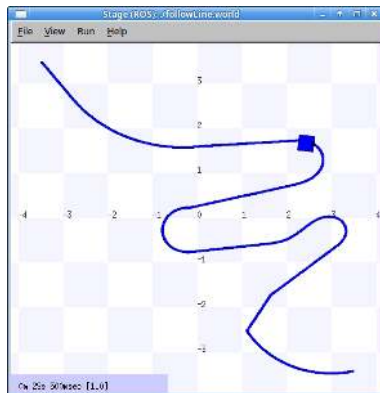
On-Off Control with
a Dead Zone

Error	Turn V.
> 0.3	-0.6
$[-0.3, 0.3]$	0
< -0.3	0.6

Forward V. = 0.5



Some Proposals for Line Following (2)

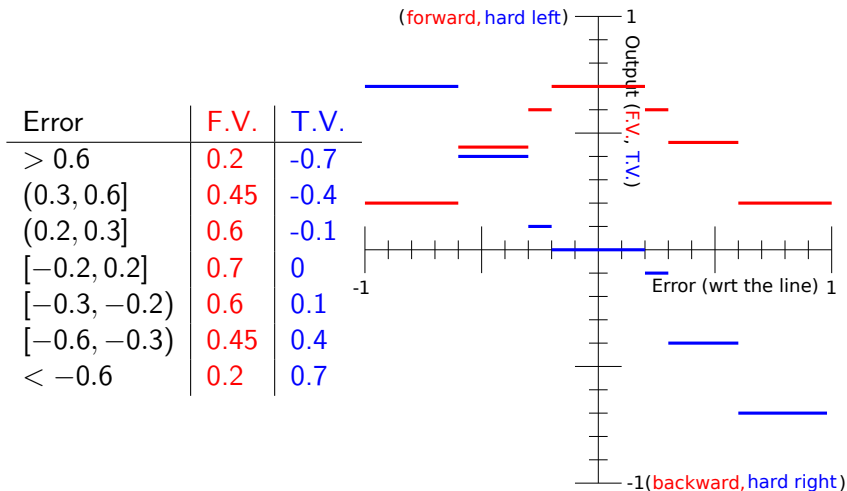


Some Proposals for Line Following (2)

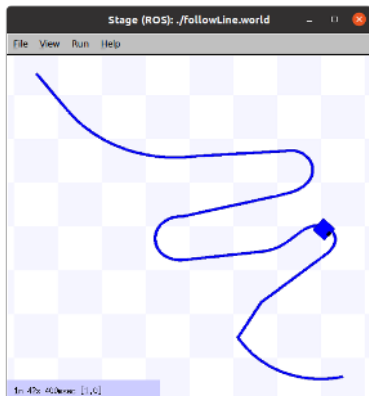
Comments:

- Maybe the velocity shouldn't be constant.
- Would more ranges of turn velocity help?

Some Proposals for Line Following (3)



Some Proposals for Line Following (3)



Some Proposals for Line Following (3)

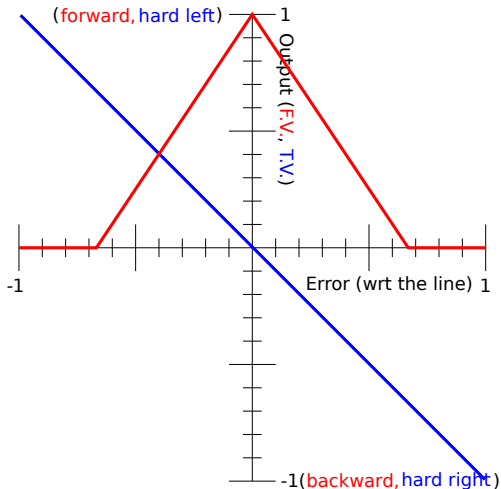
Comments:

- Maybe the turns should be more gentle when error is close to 0?
- Maybe we should stop moving forward when turning quickly (when the absolute value of the error is large).
- Couldn't we simplify all these cases with some formulas?

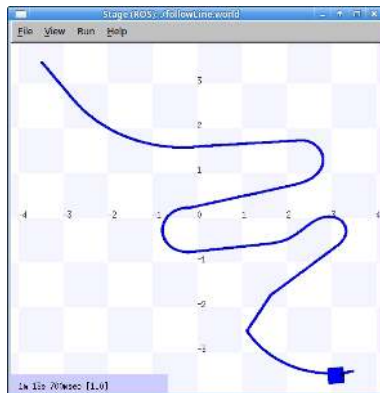
Some Proposals for Line Following (4)

$$T.V. = -1 * Error$$

$$F.V. = \max(0, 1 - \text{abs}(T.V. * 1.5))$$



Some Proposals for Line Following (4)



We will now look at more sophisticated types of feedback control.

The four most common types are:

- P - Proportional control (like our previous solution!)
Control proporcional
- PD - Proportional derivative control
Control proporcional y derivativo
- PID - Proportional integral derivative control
Control proporcional, integral y derivativo
- PI - Proportional integral control
Control proporcional e integral

Proportional Control

Proportional Control (the present): the system reacts in proportion to the error (both magnitude and 'direction').

$$\text{output} = K_p \times \text{input}$$

- This is basically what we did in the last line following example above.
- K_p is a constant which depends on the problem. For example, it changes when the way that the robot's motors respond to commands change.
- K_p is also referred to as the **gain**, as it determines the magnitude of the response.
- Determining K_p is often done by trial and error, if a good simulation exists it can be computed.

Proportional Derivative Control

Derivative Control (the future): This controller assumes that the next change in the error will be the same as the last change in the error (using the slope between those two points).

In many systems, a slightly different kind of control is needed when the system is close to its goal state. As the system approaches its goal a factor is needed to prevent the momentum of the system from over-shooting the goal.

$$\text{output} = K_p i + K_d \frac{di}{dt}$$

If dt , the sampling rate, is more or less constant it can be considered to be part of K_d .

Proportional Integral Derivative Control

Integral Controller (the past): This controller integrates (sums up) previous error and thereby compensates for them. (This helps to eliminate fixed errors.)

In some sense it keeps track of the cumulative error.

Example: systematic errors in turning.

$$output = K_p i + K_i \int i(t) dt + K_d \frac{di}{dt}$$

If dt , the sampling rate, is more or less constant it can be considered to be part of K_i .

Understanding the Terms of a PID

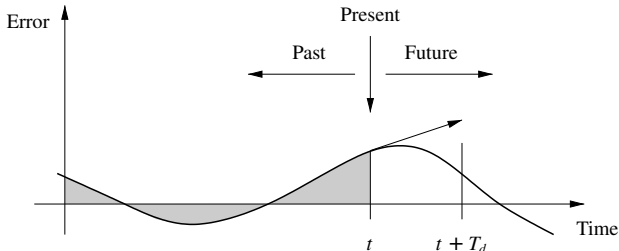


Figure 1.17: Action of a PID controller. At time t , the proportional term depends on the instantaneous value of the error. The integral portion of the feedback is based on the integral of the error up to time t (shaded portion). The derivative term provides an estimate of the growth or decay of the error over time by looking at the rate of change of the error. T_d represents the approximate amount of time in which the error is projected forward (see text).

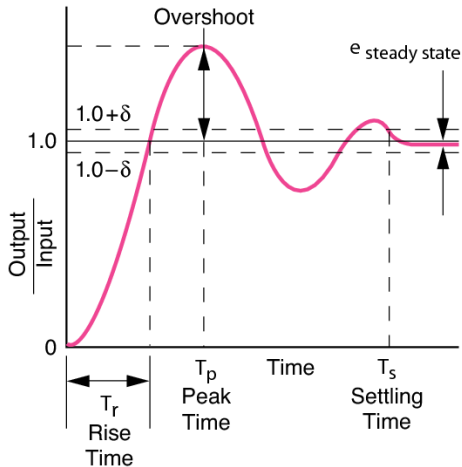
Image from *Feedback Systems* by Åström and Murray.

PID Control with Line Following

Using our line following problem as an example, and assuming a fixed dt :

- the proportional term is the distance from the center of the robot to the line
(error)
- the derivative term is how quickly the error is changing
(derivative = current error - previous error)
- the integral term is the running sum of the errors
(integral = integral + current error)

PID Parameter Tuning Terminologies



- Rise Time - *Tiempo de Levantamiento*
- Overshoot - *Sobrepaso*
- Peak Time - *Tiempo de Pico*
- Settling Time - *Tiempo de Asentamiento*
- Steady State - *Estado estacionario*

Image from <http://www.newport.com/Control-Theory-Terminology/178319/1033/content.aspx>

PID Parameter Tuning

In general, the effect of **increasing** parameters

Parameter	Rise time	Overshoot	Settling time	Steady-state error
K_p	↓	↑	Small	↓
K_i	↓	↑	↑	↓
K_d	Small	↓	↓	Small

Rise time - how quickly the system tries to fix errors

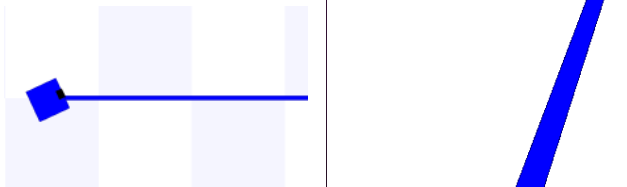
Overshoot - goal overshoot while correcting errors

Settling time - time to stabilize after large change

Steady-state error - error when the system is stable

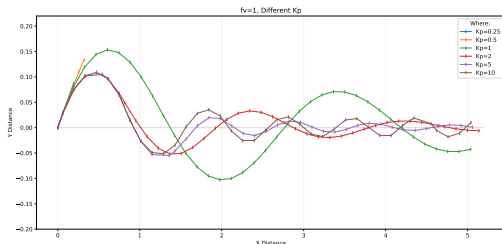
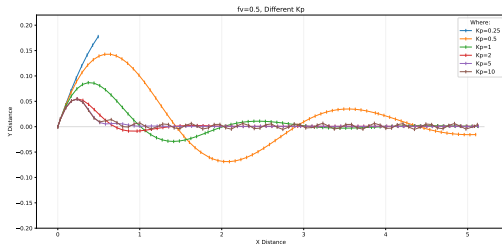
Some Testing: The World Used

A long horizontal line with the robot rotated with respect to the line at the start (a disturbance):



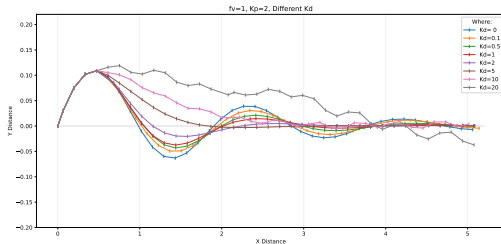
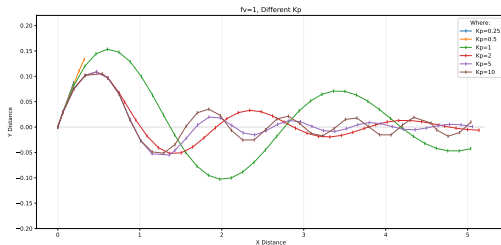
Some Testing: P Controller, Tuning by Hand

- ticks = robot position at the end of a step
- More difficult to control a robot moving faster
- With a low K_p , the robot quickly loses the line
- With a high K_p , the movement becomes a bit unstable



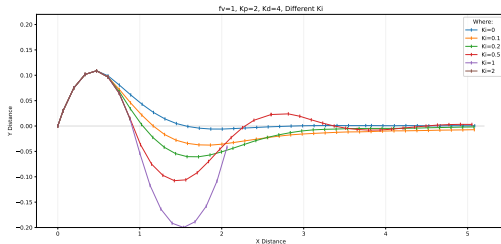
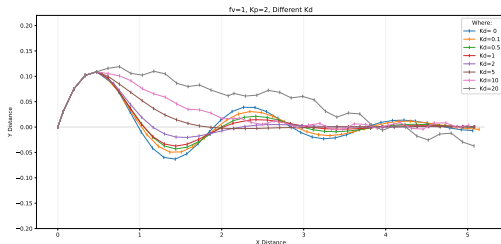
Some Testing: P and PD Controller, Tuning by Hand

- Including the derivative component smooths the curves
- A large K_d causes unstable movements



Some Testing: PD and PID Controller, Tuning by Hand

- Good performance with $K_i=0$
- Larger K_i results in greater integral contribution and eventually unstable movements (causing loss of the line)



We can use a simplification of the **Ziegler-Nichols Method** to adjust the parameters. (This is a heuristic.)

- Start by setting K_i and K_d to 0 and make a reasonable guess regarding K_p .
- If it can't reach the goal state increase K_d , if large oscillations are observed decrease K_p . You want reasonable and constant oscillations. Note the resulting K_d value as K_c (critical or ultimate gain).
- Using $K_d = K_c$ measure the oscillation period P_c .

PID Parameter Tuning

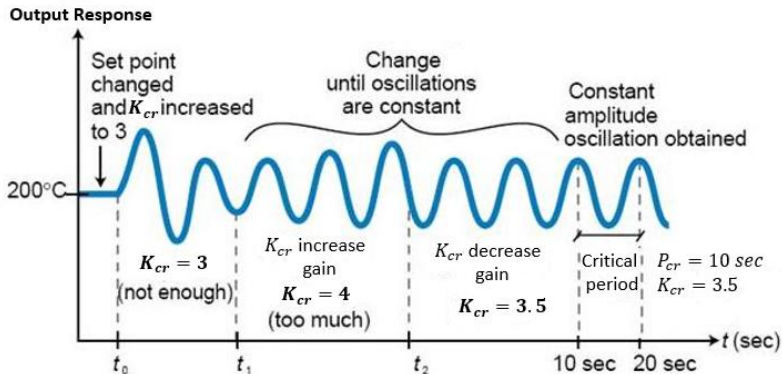


Image from <https://controls.engin.umich.edu/wiki>

PID Parameter Tuning

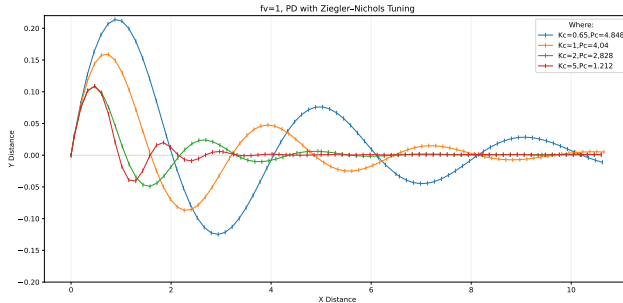
(cont.)

- Assuming a constant loop frequency, you can use the table to calculate K_p , K_i and K_d .

Type	K_p	K_i	K_d
P	$0.5K_c$		
PI	$0.45K_c$	$0.54K_p/P_c$	
PD	$0.8K_c$		$0.1K_pP_c$
PID	$0.6K_c$	$1.2K_p/P_c$	$0.075K_pP_c$

- Make small adjustments using the previous table “Effect of increasing parameters”.

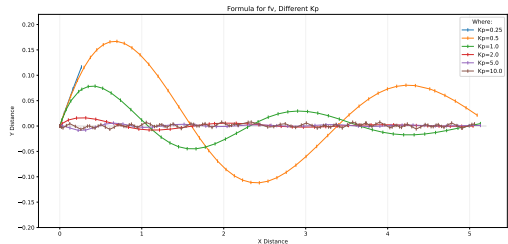
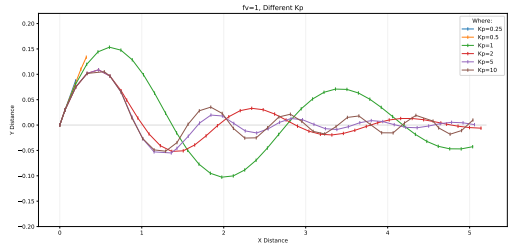
Some Testing: PD Controller, Ziegler-Nichols Tuning



Note: change of the 'X Distance' scale (previously it was 5, here it is 10).

Some Testing: P Controller with Formula for the Velocity

- Using a function for f_v might make sense, but it produces a large change in the behavior of the system



Some additional thoughts

- PID controllers are very common. Examples: ovens, cruise control, balancing robots, ...
- PID controllers are very old (they predate computer control!), they can be implemented in purely mechanical systems.
- Other more complex controllers exist
- Other (better) methods exist for tuning PID

Some additional thoughts (cont.)

- In general, the faster your loop frequency (or the slower your actuators) the easier your system is to control.
- Unless acceleration is a big issue (heavy robots, steep inclines) you may be able to eliminate the I term.
- Changing the notion of **error** results in different controllers. For example, rather than distance you can use rate of change

More complex inputs/outputs

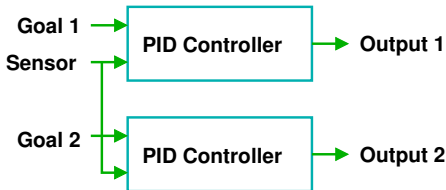
We have been discussing systems with one input and one output.

- SISO - Single input, single output



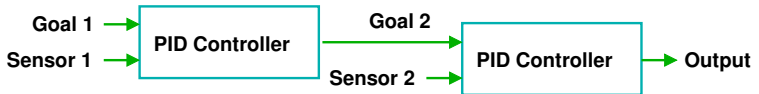
Other possibilities exist:

- SIMO - Single input, multiple output

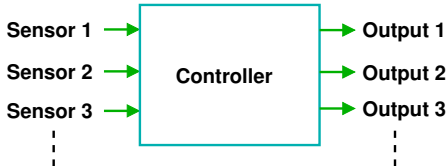


More complex inputs/outputs

- MISO - Multiple input, single output, example cascade controller



- MIMO - Multiple input, multiple output (complex, usually not PID)



The PID Controllers which we have discussed are designed for use in linear systems.

What is an example of a non-linear robot actuator?

- A motor with a non-linear relationship between input and speed.
- A motor which reacts very differently when not warmed up

In these cases **gain scheduling** can be used. Changes to K_p , K_i and K_d can be calculated based upon time or system inputs.

Another method is to combine your PID controller with a feedforward controller which compensates for certain situations (like the non-typical behavior of a system during warmup).